

Nombre(s): Alexander Anibal

Apellidos: Miranda Lino

Código: 22200121

Fecha: 20/06/20

2- I Análisis de Arquitectura y Conurrencia

1- Este patrón repositorio actúa como fachada y única fuente de verdad porque centraliza y oculta la complejidad del origen de los datos, garantizando que la interfaz de usuario consuma siempre información consistente sin saber de donde proviene.

El repositorio conecta a la UI de forma reactiva a Room para mostrar datos locales al instante, mientras que en un segundo plano descarga actualizaciones de Retrofit y las guarda en la base de datos.

2- El viewmodelscope es el ciclo de vida, que maneja las corrutinas de un ViewModel y es clave para la memoria porque este se cancela automáticamente cuando el ViewModel se destruye. Si el usuario sale a mitad de una petición de red, esta tarea se cancela automáticamente en segundo plano evitando así fugas de memoria, consumo innecesario y posibles errores.

3- En este caso, un flujo no va a ejecutar código ni emitir datos hasta tener un ~~new~~ suscriptor, creando un flujo nuevo que sea para cada uno.

Mientras que un stateflow siempre está activo, mantiene espacio de memoria. Se requiere para la UI porque almacena un valor inicial obligatorio, evita repetir tareas y retiene el último estado.

4- La inyección manual mediante Application centraliza la creación de objetos globales de infraestructura en una clase contenedor accesible desde toda la app. La palabra by lazy sirve para que estos objetos pesados no se inicialicen al arrancar la aplicación sino únicamente en el momento exacto en que se necesitan por primera vez, lo que optimiza el uso de la app.

3 - II. Sección a completar

1- El flujo de datos que emite automáticamente cada vez que una tabla de Room cambia se implementa usando el tipo Flow

2-

3- Para que un servicio en primer plano de ubicación sea válido en Android 10+, debe declararse en el manifiesto con el tipo location

4- La función applicationContext de la clase Application es la que garantiza que los componentes compartan la misma instancia del repositorio mediante un "cast".

5- El componente que permite transformar una API basada en llamadas antiguas es una función suspendible moderna se llama suspendCancellableCoroutine

6- El archivo de metadatos que describe la estructura de la app al sistema operativo se llama exactamente AndroidManifest.xml

7- En la arquitectura AOSP, la capa que actúa como puente estándar entre el framework y los controladores se denomina HAL

8- El operador de Flow que permite combinar múltiples fuentes (como GPS y Sensores) para emitir un estado unificado es combine

4: TL- Desarrollo de Código y Lógica Personalizada

Δ: Repositorio y DI

// clase que va a recibir el DAO

```
class UsuarioRepository (private val usuarioDao: UsuarioDao)
```

```
class MyApp : Application() { // instanciación
```

```
    private val database by lazy { MyDatabase.getDatabase(this) }
```

```
    val usuarioRepository by lazy {
```

```
        UsuarioRepository(database.usuarioDao())
```

```
    }
```

```
}
```


2: Lógica Dinámica de identidad

//a)

```
data class Usuario {
```

```
    val nombre: String?,
```

```
    val apellidos: String?,
```

```
    val edad: Int?
```

```
}
```

```
fun main() {
```

```
    //b
```

```
    val usuario = Usuario("Alexander", "Miranda Lino", 20)
```

```
    //c y e)
```

```
    val letrasNombre = usuario.nombre?.replace(" ", "")?
```

```
    .length ?: 0
```

```
    val letrasApellidos = usuario.apellidos?.replace(" ", "")?
```

```
    .length ?: 0
```

```
    val puntajeIdentidad = letrasNombre - letrasApellidos
```

```
    //d
```

```
    val nombreCompleto = "${usuario.nombre}?: ""}
```

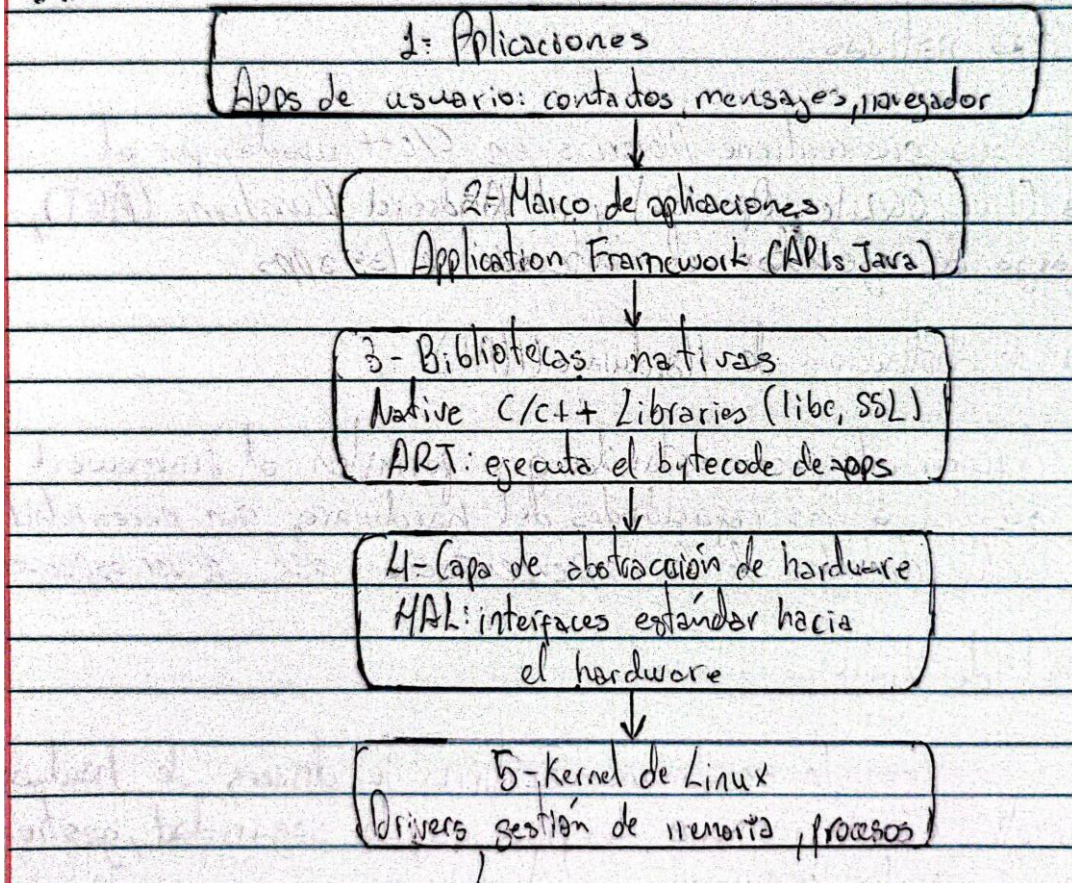
```
    "${usuario.apellidos}?: ""}", trim()
```

```
    println("Usuario: $nombreCompleto, tu puntaje de  
    identidad es: $puntajeIdentidad")
```

```
}
```


5- IV Arquitectura AOSP

3-



2-

1.- Aplicaciones

Esta es la capa superior, siendo la única visible para el usuario final. Contiene tanto las apps del sistema como la de terceros; todas se ejecutan sobre el framework.

2.- Marco de trabajo (Application Framework)

Provee las API en Java/Kotlin que los desarrolladores usan para construir apps: ActivityManager, WindowManager, etc. Es la capa que estandariza el acceso a los servicios del sistema.

3.- Bibliotecas nativas

Esta es la capa que contiene librerías en C/C++ usadas por el sistema (libc, SQLite, OpenGL) y el Android Runtime (ART). Se encarga de ejecutar el bytecode de las apps.

4.- Capa de abstracción de Hardware (HAL)

Aquí se definen interfaces estándar que permiten al framework Android acceder a las capacidades del hardware, sin necesidad de conocer los detalles de la implementación del driver específico.

5.- Kernel de Linux

Capa más baja, basada en Linux. Gestiona los drivers de hardware, la administración de memoria, los procesos, la seguridad, gestión de energía y la comunicación entre procesos.